

Análisis de Valores Extremos en Datos Sísmicos

Daniel Lara

Table of contents

0.1	Análisis de Extremos: Máximos por Bloques (GEV)	2
0.1.1	Ajuste del Modelo GEV	3
0.2	Análisis de Picos sobre el Umbral (POT)	5
0.3	Ajuste de Modelos de Excedencia (Procesos Puntuales / GPD)	6
0.3.1	Parámetros iniciales aleatorios	6
0.3.2	Con parámetros iniciales obtenidos del ajuste GEV previo:	7
0.3.3	Sin parámetros iniciales (modelo estándar GPD)	8
0.4	Diagnóstico de Independencia y Declustering	9
0.4.1	Prueba de Independencia	9
0.4.2	Diagnóstico del PP	11
0.5	Las r observaciones más grandes	12
0.6	Ajuste de parámetros con covariables	16
0.6.1	Preparación de datos	16
0.6.2	Modelo 0	17
0.6.3	Modelo 1: Tendencia temporal en la escala (σ)	18

```
# Carga de librerías
library(tidyverse)
library(sf)
library(ggplot2)
library(lubridate)
library(patchwork)
library(evd)
library(ismev)
library(evgam)
library(mgcv)
library(tseries)
```

```
library(extRemes)

# Cargar datos
datos <- readRDS("D:\\PROYECTO\\Seismic\\input\\sismicevents.rds")

datos <- datos %>%
  st_drop_geometry() %>%
  mutate(time = as.POSIXct(time, format = "%Y-%m-%d %H:%M:%S", tz = "UTC")) %>%
  arrange(time)
```

0.1 Análisis de Extremos: Máximos por Bloques (GEV)

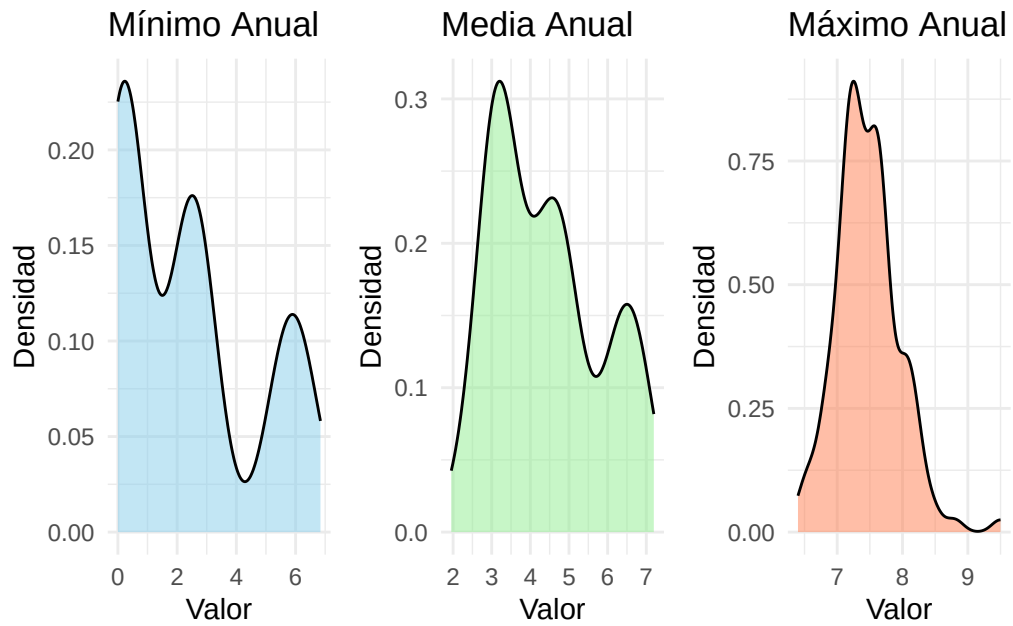
```
datos_extremos <- datos %>%
  mutate(periodo = floor_date(time, "year")) %>%
  group_by(periodo) %>%
  summarise(
    minimo = min(magnitude, na.rm = TRUE),
    media = mean(magnitude, na.rm = TRUE),
    maximo = max(magnitude, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  filter(!is.infinite(minimo) & !is.infinite(maximo))

p1 <- ggplot(datos_extremos, aes(x = minimo)) +
  geom_density(fill = "skyblue", alpha = 0.5) +
  labs(title = "Mínimo Anual", x = "Valor", y = "Densidad") +
  theme_minimal()

p2 <- ggplot(datos_extremos, aes(x = media)) +
  geom_density(fill = "lightgreen", alpha = 0.5) +
  labs(title = "Media Anual", x = "Valor", y = "Densidad") +
  theme_minimal()

p3 <- ggplot(datos_extremos, aes(x = maximo)) +
  geom_density(fill = "coral", alpha = 0.5) +
  labs(title = "Máximo Anual", x = "Valor", y = "Densidad") +
  theme_minimal()

# Mostrar Densidades
p1 + p2 + p3
```



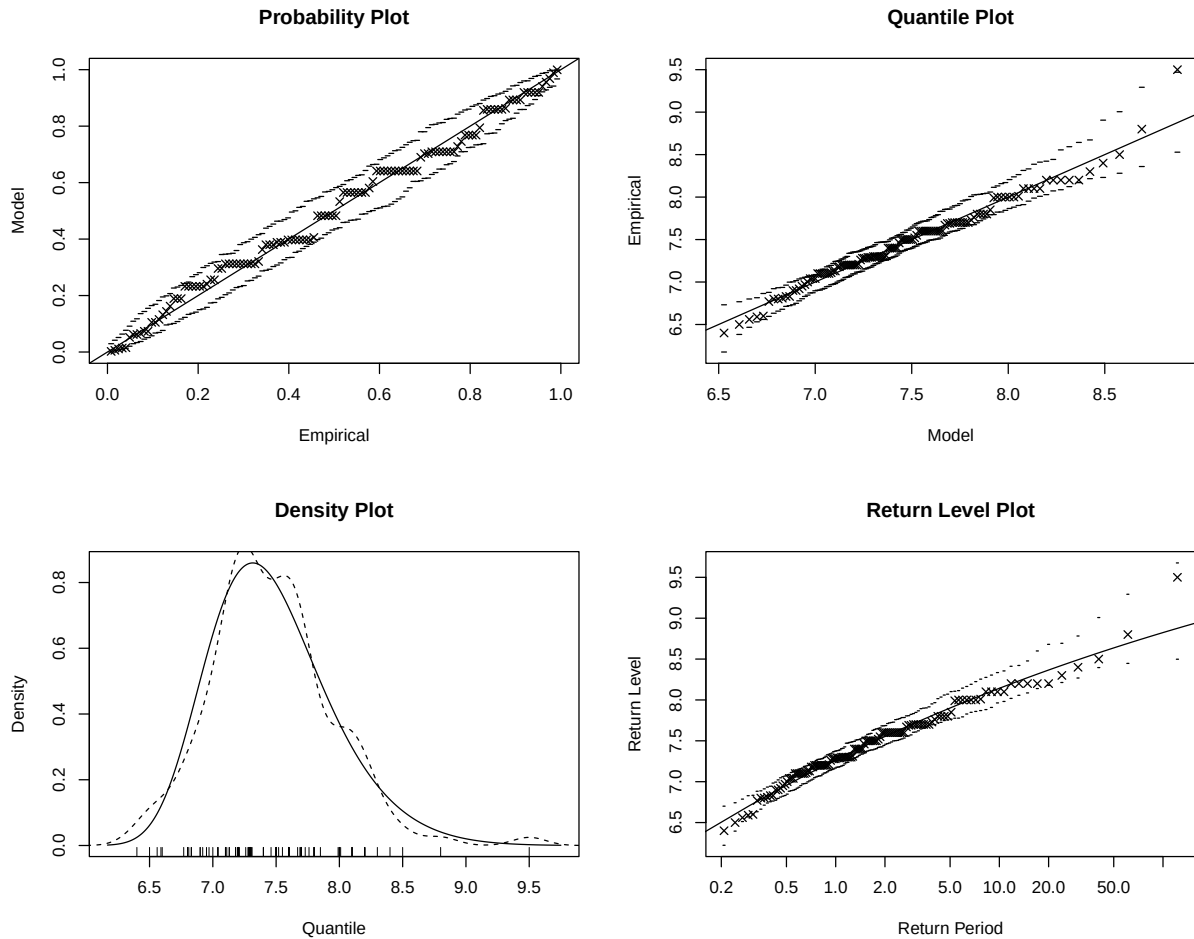
0.1.1 Ajuste del Modelo GEV

```
fit_max <- fgev(datos_extremos$maximo)

# Parámetros del modelo GEV:
fit_max$estimate
```

```
      loc      scale      shape
7.2658467 0.4306266 -0.1092916
```

```
par(mfrow = c(2, 2))
plot(fit_max)
```



```
par(mfrow = c(1, 1))
```

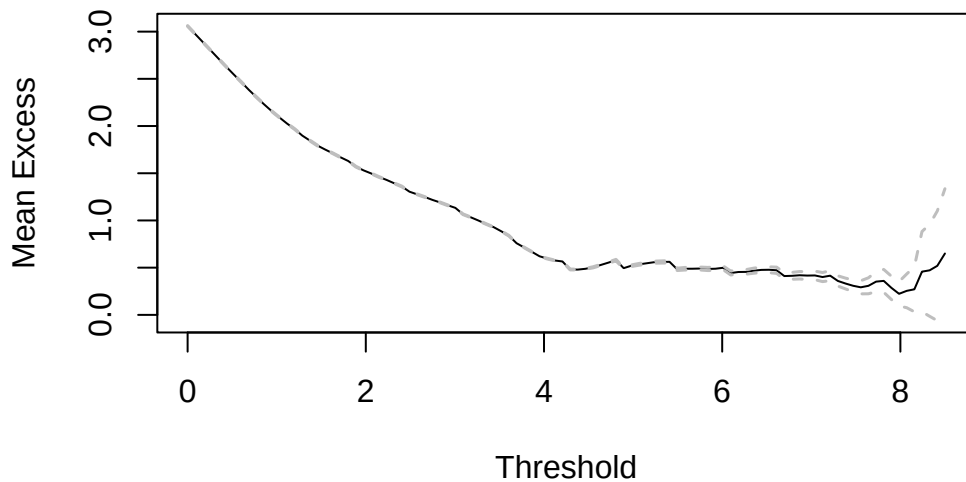
- **Gráfico de Probabilidades (P-Plot):** Podemos notar que las probabilidades reales se llegan a adaptar bien a las teóricas predichas por el modelo GEV.
- **Gráfico de Cuantiles (Q-Plot):** Las magnitudes van de 6 a 9, notamos que ajusta bien los valores más probables, sin embargo, en los extremos predichos ya se alejan de los datos reales y aumenta la incertidumbre.
- **Nivel de Retorno:** Nos muestra que los sismos entre 6 a 7.5 ocurren en periodos de 0.5 a 5 años; sin embargo, los sismos de magnitud 8, por ejemplo, ocurren cada 50 años.

Máxima de estabilidad por los hablado anteriormente, los datos se llegan a ajustar bien a un modelo GEV, por lo tanto, decimos que es la única distribución de los datos que es máxima estable Extremal Types Theorem y dado que el valor de $\xi_i < 0$, estamos en una GEV reversed Weibull

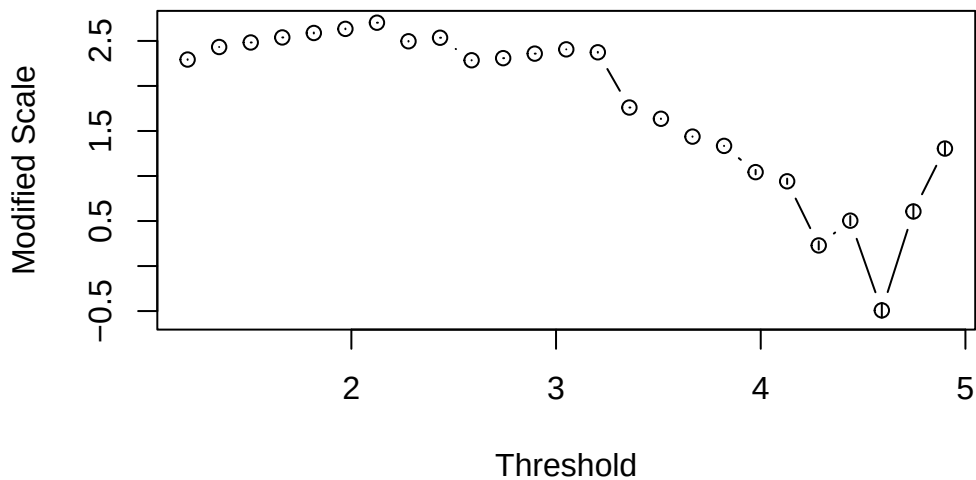
0.2 Análisis de Picos sobre el Umbral (POT)

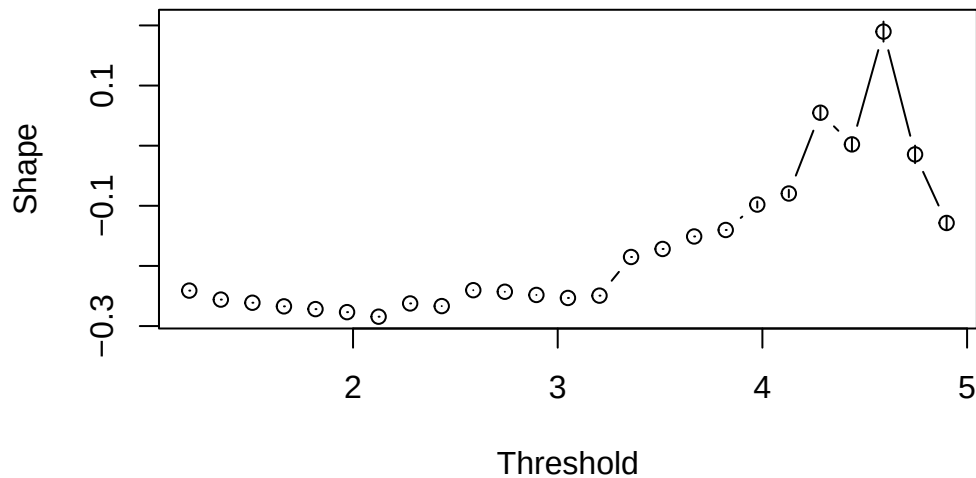
```
# Gráfico de Vida Media Remanente  
mrlplot(datos$magnitudo, main="Gráfico de Vida Media Remanente para Sismos")
```

Gráfico de Vida Media Remanente para Sismos



```
# Gráfico de elección de umbral (tcplot)  
limites_umbral <- quantile(datos$magnitudo, probs = c(0.05, 0.95))  
tcplot(datos$magnitudo, tlim=limites_umbral)
```





```
UMBRAL <- 5.8
```

Se toma un umbral entre 5.5 a 6

Se evidencia posiblemente que el parámetro $\xi < 0$ mostrando que las colas serán ligeras con un límite máximo

0.3 Ajuste de Modelos de Excedencia (Procesos Puntuales / GPD)

0.3.1 Parámetros iniciales aleatorios

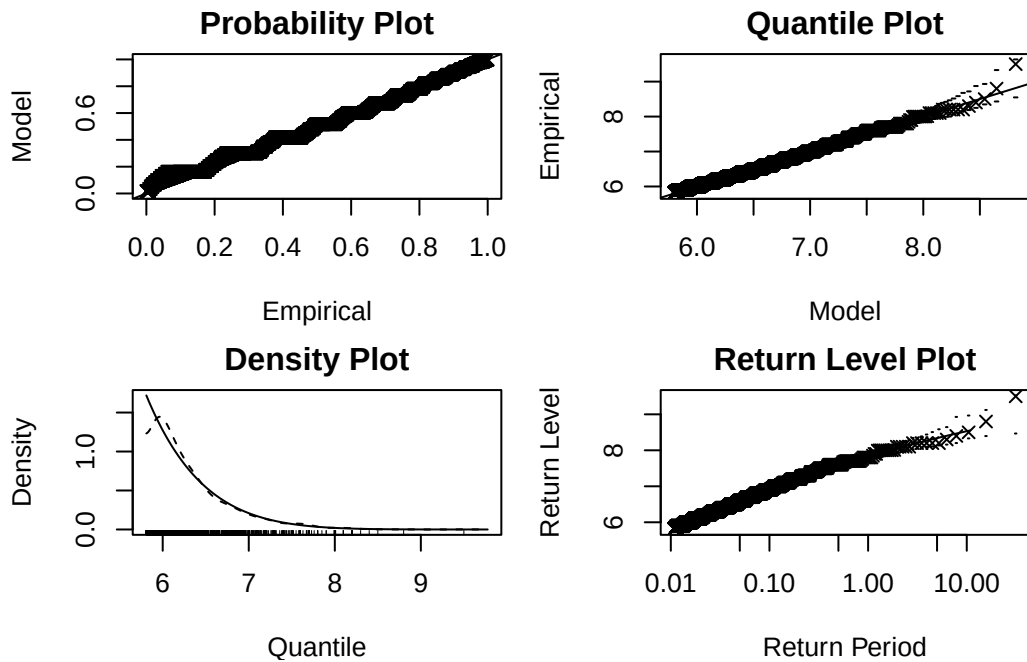
```
fit_rd <- fpot(datos$magnitude, threshold=5.8, model="pp",
               start=list(loc=10, scale=1, shape=0.1), npp=365.25*24)
fit_rd$estimate
```

```
      loc      scale      shape
7.831274 0.344794 -0.112154
```

```
fit_rd$std.err
```

```
      loc      scale      shape
0.04317405 0.01777771 0.01454244
```

```
par(mfrow=c(2,2), mar=c(4,4,2,1))
plot(fit_rd)
```



El npp es correcto porque son una recopilación horaria por cada día del año

0.3.2 Con parámetros iniciales obtenidos del ajuste GEV previo:

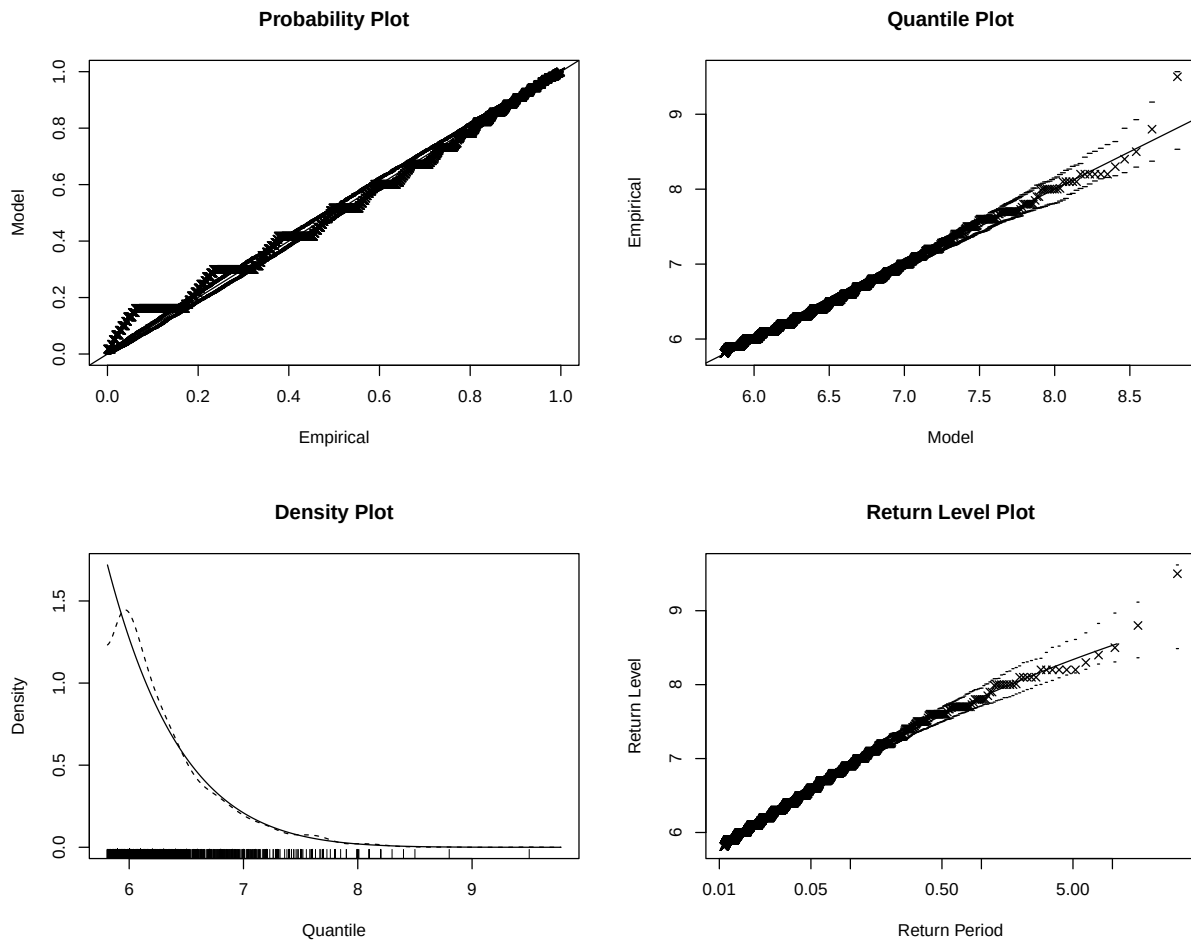
```
p_egv <- fit_max$estimate
fit_ex <- fpot(datos$magnitude, threshold=5.8, model="pp",
              start=as.list(p_egv), npp=365.25*24)
fit_ex$estimate
```

```
      loc      scale      shape
7.8312785 0.3447975 -0.1121548
```

```
fit_ex$std.err
```

```
      loc      scale      shape
0.04318605 0.01778548 0.01454822
```

```
par(mfrow=c(2,2))
plot(fit_ex)
```



```
par(mfrow=c(1,1))
```

0.3.3 Sin parámetros iniciales (modelo estándar GPD)

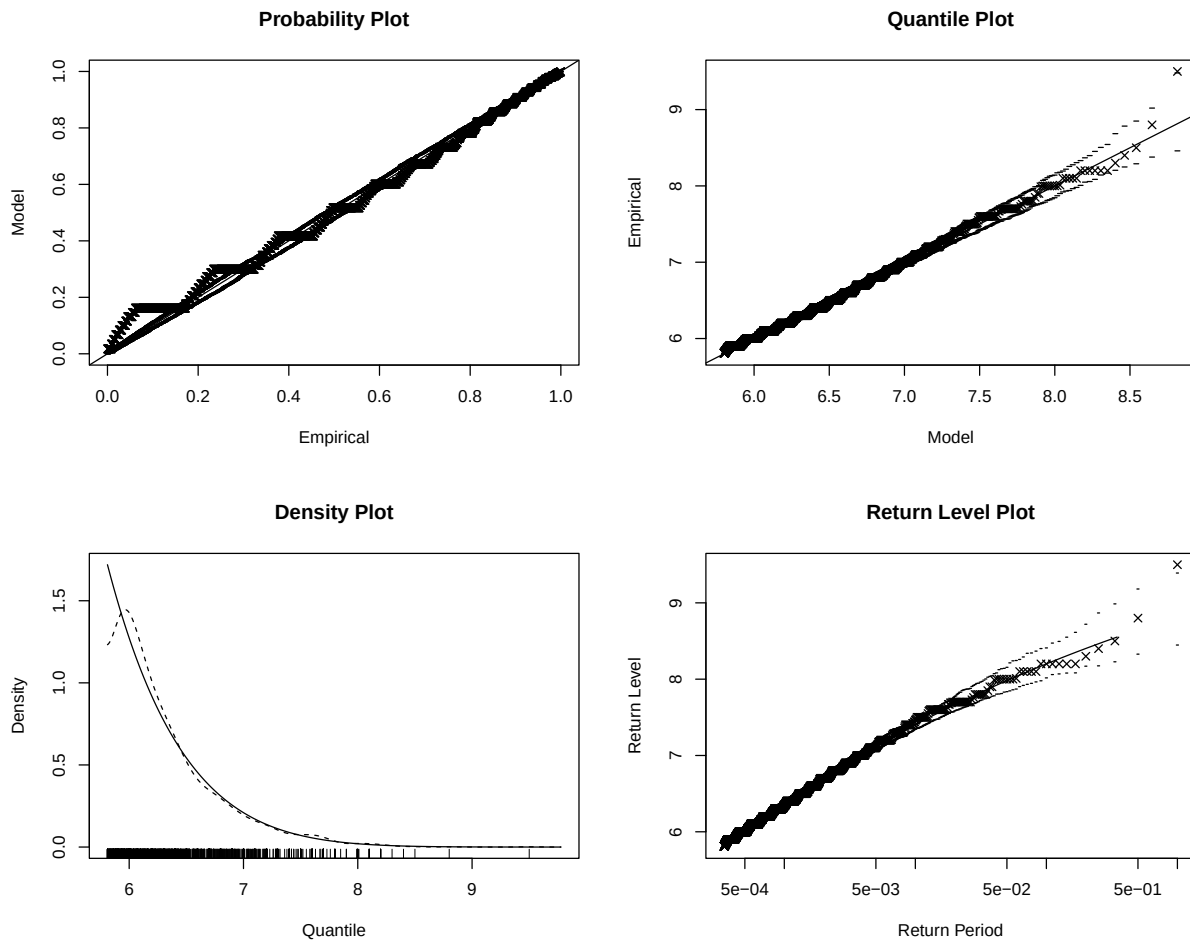
```
fit_np <- fpot(datos$magnitude, threshold = 5.8)
fit_np$estimate
```

```
      scale      shape
0.5726232 -0.1122663
```

```
fit_np$std.err
```

```
scale      shape  
0.01347679 0.01454875
```

```
par(mfrow=c(2,2))  
plot(fit_np)
```



```
par(mfrow=c(1,1))
```

0.4 Diagnóstico de Independencia y Declustering

0.4.1 Prueba de Independencia

```

excedencias_binarias <- as.numeric(datos$magnitude > UMBRAL)
#H0: son independientes los sismos
runs_test <- runs.test(factor(excedencias_binarias))

cat("=== PRUEBA DE RACHAS PARA INDEPENDENCIA ===\n")

```

=== PRUEBA DE RACHAS PARA INDEPENDENCIA ===

```
cat("Estadístico:", round(runs_test$statistic, 4), "\n")
```

Estadístico: -145.6154

```
cat("p-valor:", round(runs_test$p.value, 4), "\n\n")
```

p-valor: 0

#<0.05 se rechaza la hipotesis nula

```

# Filtrar excedencias para declustering
excedencias <- datos %>%
  filter(magnitude > UMBRAL) %>%
  arrange(time)

# Aplicar declustering por método de rachas
dl_object <- decluster(
  x = excedencias$magnitude,
  threshold = UMBRAL,
  method = "runs",
  r = 3 # Número de observaciones bajo el umbral para cerrar un cluster
)

# Crear dataset con eventos declusterizados
excedencias <- excedencias %>%
  mutate(
    cluster_id = dl_object,
    es_evento_principal = !is.na(cluster_id)
  )

# Separar datasets

```

```

excedencias_original <- excedencias # Todos los eventos
excedencias_declust <- excedencias %>%
  filter(es_evento_principal == TRUE) # Solo eventos principales

cat("Número de eventos después del declustering:", nrow(excedencias_declust), "\n")

```

Número de eventos después del declustering: 2885

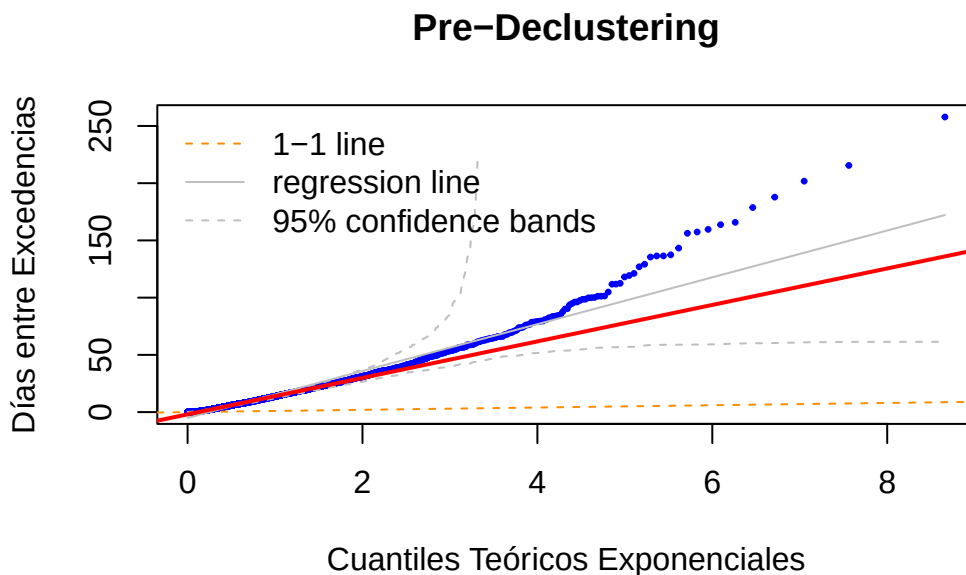
0.4.2 Diagnóstico del PP

```

# Diagnóstico PRE-declustering
tiempos_pre <- diff(as.numeric(excedencias_original$time)) / (3600 * 24)

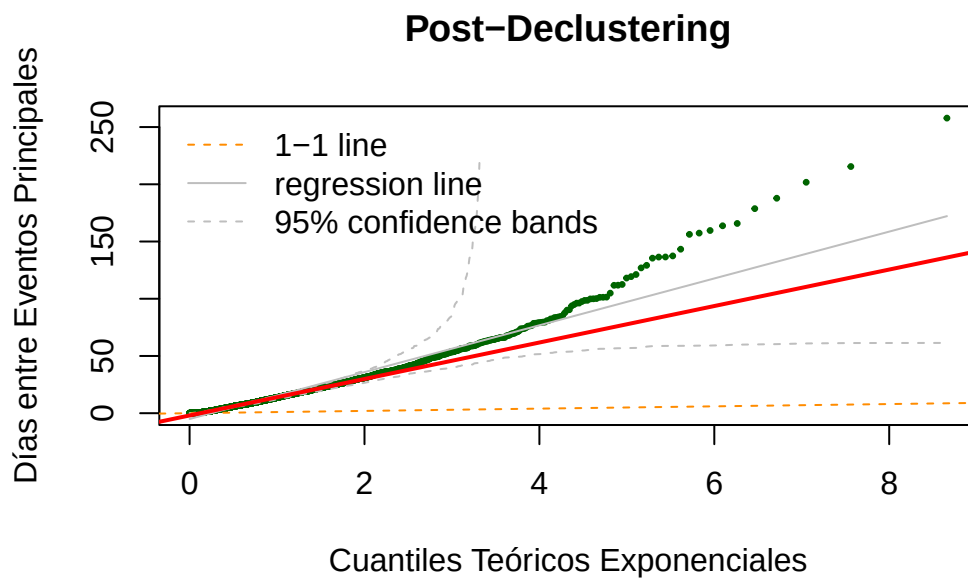
qqplot(
  qexp(ppoints(length(tiempos_pre))),
  tiempos_pre,
  main = "Pre-Declustering",
  xlab = "Cuantiles Teóricos Exponenciales",
  ylab = "Días entre Excedencias",
  pch = 20, col = "blue", cex = 0.5
)
qqline(tiempos_pre, distribution = qexp, col = "red", lwd = 2)

```



```
# Diagnóstico POST-declustering
tiempos_post <- diff(as.numeric(excedencias_declust$time)) / (3600 * 24)

qqplot(
  qexp(ppoints(length(tiempos_post))),
  tiempos_post,
  main = "Post-Declustering",
  xlab = "Cuantiles Teóricos Exponenciales",
  ylab = "Días entre Eventos Principales",
  pch = 20, col = "darkgreen", cex = 0.5
)
qqline(tiempos_post, distribution = qexp, col = "red", lwd = 2)
```



por qué se abren los IC? si ya eliminamos las replicas

0.5 Las r observaciones más grandes

Cómo definimos el ancho de cada bloque? Cómo definirlo en función del umbral (u)

Definiré el r como:

$$r \sim \frac{\# \text{ de sismos que superar el umbral } u}{\text{Número total de bloques (años?)}}$$

```

num_exc <- sum(datos$magnitude >= UMBRAL, na.rm = TRUE)
num_anios <- length(unique(year(datos$time)))
r_optimo <- trunc(num_exc / num_anios)

cat("Número de excedencias totales:", num_exc, "\n")

```

Número de excedencias totales: 3264

```

cat("Número de años:", num_anios, "\n")

```

Número de años: 122

```

cat("r óptimo calculado:", r_optimo, "\n")

```

r óptimo calculado: 26

```

# Preparar matriz para r-largest
bloques <- datos %>%
  mutate(year = year(time)) %>%
  group_by(year) %>%
  filter(n() >= r_optimo) %>%
  slice_max(order_by = magnitude, n = r_optimo, with_ties = FALSE) %>%
  mutate(r_orden = row_number()) %>%
  ungroup()

# Crear matriz en formato wide
matriz_r <- bloques %>%
  select(year, r_orden, magnitude) %>%
  pivot_wider(
    names_from = r_orden,
    values_from = magnitude,
    names_prefix = "r_"
  ) %>%
  select(-year) %>%
  as.matrix()

cat("Número de bloques (años):", nrow(matriz_r), "\n")

```

Número de bloques (años): 101

```
cat("Número de observaciones por bloque:", ncol(matriz_r), "\n")
```

Número de observaciones por bloque: 26

```
# Ajuste de modelos r-largest y comparación
resultados_r <- data.frame(
  Modelo = character(),
  r = integer(),
  NLLH = numeric(),
  AIC = numeric(),
  Convergencia = integer(),
  stringsAsFactors = FALSE
)

for(i in 2:min(7, ncol(matriz_r))) {
  fit_temp <- try(rlarg.fit(matriz_r[, 1:i, drop = FALSE]), silent = TRUE)

  if (!inherits(fit_temp, "try-error") && fit_temp$conv == 0) {
    aic_temp <- 2 * fit_temp$nlh + 2 * length(fit_temp$mle)
    resultados_r <- rbind(resultados_r, data.frame(
      Modelo = paste("r-largest (r =", i, ")"),
      r = i,
      NLLH = round(fit_temp$nlh, 2),
      AIC = round(aic_temp, 2),
      Convergencia = fit_temp$conv,
      stringsAsFactors = FALSE
    ))
  }
}
```

```
# Ordenar por AIC
resultados_r <- resultados_r[order(resultados_r$AIC), ]
cat("\n=== TABLA COMPARATIVA MODELOS r-largest ===\n")
```

=== TABLA COMPARATIVA MODELOS r-largest ===

```
print(resultados_r, row.names = FALSE)
```

Modelo	r	NLLH	AIC	Convergencia
--------	---	------	-----	--------------

```

r-largest (r = 7 ) 7 -552.13 -1098.26      0
r-largest (r = 6 ) 6 -393.27  -780.53      0
r-largest (r = 5 ) 5 -265.69  -525.37      0
r-largest (r = 4 ) 4 -127.53  -249.07      0
r-largest (r = 3 ) 3  -11.96   -17.92      0
r-largest (r = 2 ) 2   42.04    90.08      0

```

```

# Identificar mejor modelo
mejor_r <- resultados_r$r[1]
cat("\nMejor modelo: r =", mejor_r, "con AIC =", resultados_r$AIC[1], "\n")

```

Mejor modelo: r = 7 con AIC = -1098.26

```

invisible(capture.output({
  fit_mejor <- rlarg.fit(matriz_r[, 1:mejor_r, drop = FALSE])
}))

cat("=== PARÁMETROS ESTIMADOS DEL MEJOR MODELO ===\n\n")

```

=== PARÁMETROS ESTIMADOS DEL MEJOR MODELO ===

```
print(round(fit_mejor$mle, 4))
```

```
[1]  7.3287  0.4226 -0.1115
```

```
cat("\nErrores estándar:\n")
```

Errores estándar:

```
print(round(fit_mejor$se, 4))
```

```
[1] 0.0340 0.0175 0.0274
```

0.6 Ajuste de parámetros con covariables

EVGAM busca estimar los parámetros μ, σ, ξ por medio de combinaciones lineales con funciones llamadas B-splines que dependen de covariables (tiempo, magnitud, etc)

$$\mu(t) = \sum_{j=1}^k \beta_j b_j(t)$$

- $b_j(t)$ son las funciones base para estimar la función suave que describe la tendencia
- β_j parámetros libres que ajustan la Máxima Verosimilitud Penalizada (la forma como ajusta los parámetros)

Y trata de acercar los parámetros maximizando la log-verosimilitud penalizada

$$L_p(\beta) = l(\beta) - \frac{1}{2} \sum_j \lambda_j \beta^\top S_j \beta$$

- $l(\beta)$: Es la log-verosimilitud convencional (la bondad de ajuste).
- λ_j : Es el **parámetro de suavizado** (smoothing parameter). Es la perilla que controla qué tanto castigamos la falta de suavidad. (grande muy suave, pequeño curva muy flexible, tal vez sobreajustada)
- $\hat{S}_j$: Es una medida de la “curvatura” o rugosidad de la función. Si la línea sube y baja bruscamente, este término es muy grande.

0.6.1 Preparación de datos

```
excedencias_evgam <- excedencias_declust %>%  
  mutate(  
    excess = magnitud - UMBRAL,  
    year_decimal = year(time) + (yday(time) / 365.25),  
    time_days = as.numeric(difftime(time, min(time), units = "days"))  
  ) %>%  
  as.data.frame()
```

Se toma la diferencia con el mínimo para tener un tiempo inicial dentro de los datos que exceden el umbral

0.6.2 Modelo 0

```
fit_evgam0 <- evgam(  
  formula = list(excess ~ 1, ~ 1),  
  data = excedencias_evgam,  
  family = "gpd"  
)  
  
cat("=== MODELO 0: GPD ESTACIONARIO ===\n")
```

=== MODELO 0: GPD ESTACIONARIO ===

```
summary(fit_evgam0)
```

**** Parametric terms ****

logscale

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.56	0.02	-23.69	<2e-16

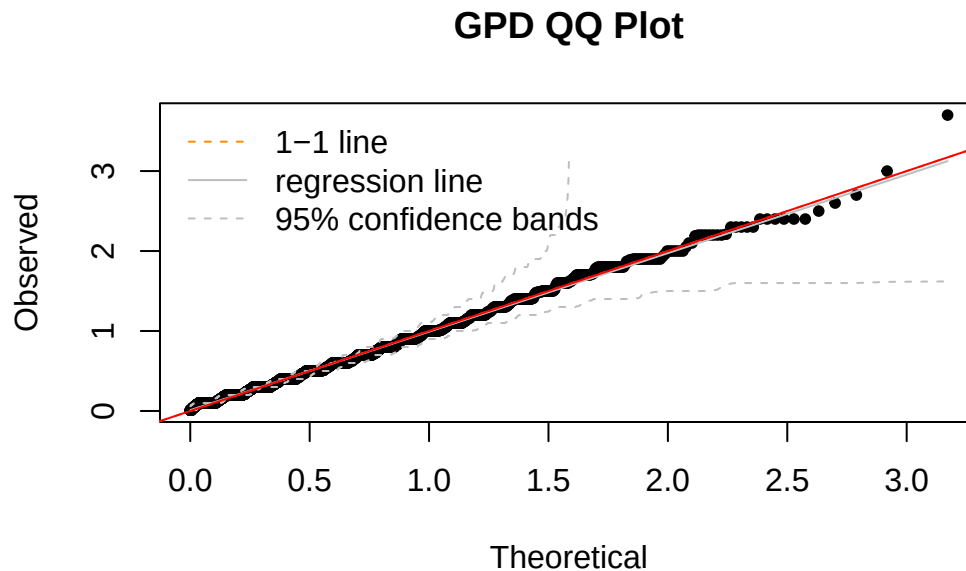
shape

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.11	0.01	-7.71	6.05e-15

**** Smooth terms ****

```
pred_par <- predict(fit_evgam0, type = "response")  
sigma_hat <- as.numeric(pred_par[, 1])  
xi_hat <- as.numeric(pred_par[, 2])  
y <- as.numeric(excedencias_evgam$excess)  
# Empirical probabilities  
p <- ppoints(length(y))  
# Theoretical GPD quantiles  
q_theo <- ifelse(  
  abs(xi_hat) > 1e-8,  
  sigma_hat / xi_hat * ((1 - p)^(-xi_hat) - 1),  
  -sigma_hat * log(1 - p)  
)
```

```
qqplot(q_theo, y,
       main = "GPD QQ Plot",
       xlab = "Theoretical",
       ylab = "Observed")
abline(0, 1, col = "red")
```



por qué se abren los intervalos?

0.6.3 Modelo 1: Tendencia temporal en la escala (σ)

fit_1

```
fit_evgam1 <- evgam(
  formula = list(excess ~ s(year_decimal, bs = "cs"), ~ 1),
  data = excedencias_evgam,
  family = "gpd"
)
```

```
pred_par1 <- predict(fit_evgam1, type = "response")
sigma_hat1 <- as.numeric(pred_par1[, 1])
xi_hat1 <- as.numeric(pred_par1[, 2])
# Theoretical GPD quantiles
q_theo <- ifelse(
  abs(xi_hat1) > 1e-8,
  sigma_hat1 / xi_hat1 * ((1 - p)^(-xi_hat1) - 1),
```

```

    -sigma_hat1 * log(1 - p)
  )

qqplot(q_theo, y,
       main = "GPD QQ Plot",
       xlab = "Theoretical",
       ylab = "Observed")
abline(0, 1, col = "red")

```

GPD QQ Plot

